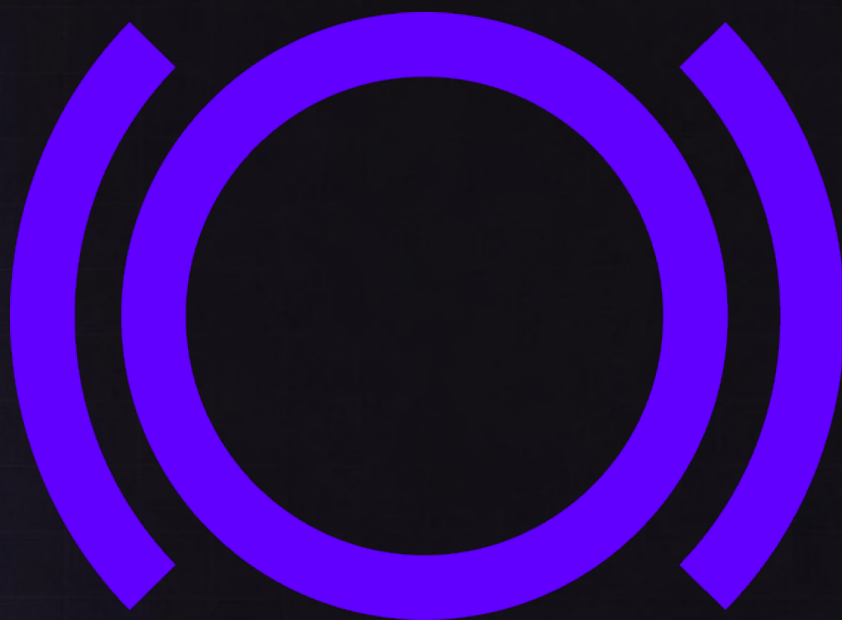


Panoptic

Security Review



March 20, 2026

Prepared by: Valves Security

Lead Auditors: Vesko210 | Merulez99

Contents

1	About Valves Security	3
2	Disclaimer	3
3	Risk Classification	3
4	Protocol Summary	4
5	Executive Summary	4
6	Findings	5
	High Findings	5
	[H-01] Inconsistent spot/TWAP pricing basis in computeNAV() makes NAV inaccurate .	5
	Low/Info Findings	6
	[L-01] Users can not create their desired vault because of createVault front-running	6
	[L-02] Performance Fee Bypassed by Cross-Address PnL Netting via Share Transfer . .	8
	[L-03] Permissionless executeWithdrawal can front-run user opt-out and force redeposit	9
	[I-01] Uninitialized feeWallet causes partial withdrawal DoS	9

1 About Valves Security

Valves Security is a Web3 security team founded by Veselin and Valeri. With deep experience across EVM-based protocols, Cosmos (Go) chains, and native Rust systems, we help projects identify and eliminate critical vulnerabilities. With a strong background and a strong team of freelancers we can assure a top quality audit.

Book a Security Review at valvessecurity.com or message us on X Valves Security

2 Disclaimer

The Valves team makes all effort to find as many vulnerabilities in the code in the given time period. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the implementation. We stand by our process, yet encourage follow-up audits and meaningful bug bounty incentives for continued whitehat protection.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- High - leads to a significant material loss of assets in the protocol or a group of users
- Medium - leads to a moderate material loss of assets in the protocol, moderately harms a group of users or to some disruption of the protocol's functionality
- Low - No funds at risk

Likelihood

- High - attack path is possible with reasonable assumptions. The issue is almost certain to happen.
- Medium - only a conditionally incentivized attack vector, but still relatively likely

- Low - has unlikely assumptions or requires a significant stake by the attacker

4 Protocol Summary

Panoptic is a decentralized derivatives protocol that introduces perpetual, oracle-free options on top of existing AMM liquidity. By leveraging Uniswap v3 liquidity positions as the foundation for options, the protocol enables permissionless minting, trading, and market-making of put and call options on any token pair-without intermediaries, expiries, or external price feeds. This design transforms liquidity into programmable volatility exposure, creating a capital-efficient, composable options primitive that aligns on-chain pricing with real market activity.

5 Executive Summary

Overview

Project	Panoptic
Commit Hash	dd51c7f0bdb0016c18dd93b61c1e48aaae60c204
Timeline	17/03/2026 – 20/03/2026

Scope

Id	Files in scope
1	PanopticVaultAccountant.sol, HypoVault.sol, HypoVaultFactory.sol

Issues Found

Critical Risk	0
High Risk	1
Medium Risk	0
Low/Info Risk	4

6 Findings

High Findings

[H-01] Inconsistent spot/TWAP pricing basis in computeNAV() makes NAV inaccurate

Description `PanopticVaultAccountant.computeNAV()` uses two different price bases within the same valuation path.

For non-credit legs (width > 0), the accountant derives the token0/token1 composition of each liquidity chunk using the pool's current spot tick:

```
1 pool.getCurrentTick() -> Math.getAmountsForLiquidity(...)
```

However, later in the same function, the resulting exposures are converted into the vault's underlyingToken using the pool TWAP obtained from `pool.getTWAP()`.

As a result, the accountant is not performing a coherent valuation under a single price basis. It composes the position using spot, but values that composition using TWAP. This creates a manipulable pricing surface during the only two lifecycle points where NAV matters for user settlement:

- `fulfillDeposits()`, where NAV determines how many shares depositors receive
- `fulfillWithdrawals()`, where NAV determines how many assets withdrawing users receive

An attacker can exploit this by sandwiching the manager's `fulfillDeposits()` or `fulfillWithdrawals()` transaction. By moving the pool spot tick immediately before the fulfillment transaction executes, the attacker can distort the token0/token1 split returned by `getAmountsForLiquidity()`. If the TWAP deviation checks still pass, the accountant will accept the distorted composition and convert it using TWAP, producing an inaccurate NAV.

The issue is not that spot is used, or that TWAP is used, but that both are used simultaneously for different parts of the same valuation flow.

Attack scenario

Setup: Vault holds a `LINK/UNI` Panoptic pool position. Vault has deposited 100 LINK as collateral into `collateralToken0`. Manager is preparing to call `fulfillDeposits()` to settle pending depositors.

Block N - Attacker front-runs:

1. Attacker executes a large swap on the LINK/UNI Uniswap pool - sells massive LINK, buying UNI
2. Spot crashes: tick 23,027 → 13,863 (LINK drops from 10 UNI to 4 UNI)
3. TWAP barely moves: still ~22,885 (EMA hasn't updated yet - sameblock)

Block N - Manager's `fulfillDeposits()` executes (same block, next tx):

4. Manager reads `pool.getTWAP()` = 22,885 → submits this as `poolPrice` in `managerInput`
5. `computeNAV()` runs:
 - `pool.getCurrentTick()` = 13,863 (spot, crashed)
 - `getAmountsForLiquidity(13863, chunk)` → position is below range [18000, 26760] → all token0 (LINK)

- `previewRedeem()` → 100 LINK collateral
 - `pool.getTWAP()` = 22,885 → stale
 - Deviation check: $|22885 - 22885| = 0 \leq 1000$ → passes
 - Conversion: $100+ \text{LINK} \times \text{TWAP_price} (9.86 \text{ UNI/LINK}) = \sim 986 \text{ UNI}$ reported
 - Real value: $100 \text{ LINK} \times \text{spot} (4 \text{ UNI/LINK}) = \sim 400 \text{ UNI}$ actual
6. Depositors in this batch receive shares based on NAV = 986 UNI - 2.46× fewer shares than they should

Block N - Attacker back-runs:

7. Attacker swaps back, restoring spot price and collecting the Uniswap fee arbitrage profit
- Why the deviation check doesn't stop this:
 The check at L295 is `|managerPrice - twapTick|`. The manager honestly reads TWAP (22,885) and submits it. TWAP is also 22,885. Deviation = 0. The check was designed to prevent a malicious manager from submitting an arbitrary price - it was not designed to detect TWAP staleness relative to spot. There is no `|getCurrentTick() - twapTick|` check anywhere in the function.

Profit source for the attacker: If the attacker is a withdrawer (not a depositor), the inflated NAV also means `fulfillWithdrawals()` pays them more UNI per share than the vault is actually worth. In the inflated-NAV direction, the attacker can front-run a withdrawal batch instead, receive excess UNI, then back-run to restore the price - extracting value directly from the vault's assets.

Recommended mitigation Use a single pricing basis throughout NAV computation.

Use `twapTick` as the single price source for both amount computation and conversion. Replace the `getCurrentTick()` call at L199 with `managerPrices[i].poolPrice`, which is already validated against TWAP by the `StaleOraclePrice` guard:

If the intended design is to keep spot-based composition, then add an explicit bound ensuring `getCurrentTick()` cannot materially deviate from TWAP before using it in `getAmountsForLiquidity()`.

Low/Info Findings

[L-01] Users can not create their desired vault because of `createVault` front-running

Description

`HypoVaultFactory.createVault()` deploys vault clones using `Clones.cloneDeterministic()` with a caller-supplied `salt` that does not incorporate `msg.sender`:

```
1 // HypoVaultFactory.sol:68
2 vault = payable(Clones.cloneDeterministic(hypoVaultReference, salt));
```

Because the deterministic address is derived from `keccak256(0xff ++ factory ++ salt ++ initCodeHash)`, and `msg.sender` plays no role in the salt, every caller competes for the same salt namespace. An attacker observing a pending `createVault` transaction in the mempool can extract the salt and front-run the victim with identical salt but attacker-controlled parameters - most critically, `manager`.

Impact

The attacker deploys a vault at the exact address the victim expected their vault to occupy, initialised with a `manager` the attacker controls. The victim's transaction reverts because the address of the vault is already deployed.

```
1 // HypoVault.sol:619-625
2 function manage(
3     address target,
4     bytes calldata data,
5     uint256 value
6 ) external onlyManager returns (bytes memory result) {
7     result = target.functionCallWithValue(data, value);
8 }
```

Proof of Concept

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.28;
3
4 import "forge-std/Test.sol";
5 import {HypoVaultFactory} from "src/HypoVaultFactory.sol";
6 import {HypoVault} from "src/HypoVault.sol";
7 import {IVaultAccountant} from "src/interfaces/IVaultAccountant.sol";
8 import {Clones} from "lib/openzeppelin-contracts-upgradeable/lib/openzeppelin-
contracts/contracts/proxy/Clones.sol";
9 import {Errors as OZErrors} from "lib/openzeppelin-contracts-upgradeable/lib/openzeppelin-
contracts/contracts/utils/Errors.sol";
10
11 contract SaltFrontrunTest is Test {
12     HypoVaultFactory factory;
13     address vaultImpl;
14
15     address victim = makeAddr("victim");
16     address attacker = makeAddr("attacker");
17     address token = makeAddr("token");
18     IVaultAccountant accountant = IVaultAccountant(makeAddr("accountant"));
19
20     bytes32 constant SALT = bytes32(uint256(0xdeadbeef));
21
22     function setUp() public {
23         vaultImpl = address(new HypoVault());
24         factory = new HypoVaultFactory(vaultImpl);
25     }
26
27     function test_saltFrontrun() public {
28         // 1. Pre-compute the address the victim expects their vault at.
29         address expectedVault = Clones.predictDeterministicAddress(factory,
            hypoVaultReference(), SALT, address(factory));
30
31         // 2. Attacker front-runs with the same salt but their own manager address.
32         vm.prank(attacker);
33         address deployedVault = factory.createVault(
34             token,
35             attacker, // <-- attacker becomes manager
36             accountant,
37             500,
38             "avUSDC",
39             "Attacker Vault USDC",
40             SALT
41         );
42
43         // 3. Victim's transaction now reverts -- address already taken.
44         vm.prank(victim);
45         vm.expectRevert(abi.encodeWithSelector(OZErrors.FailedDeployment.selector));
46         factory.createVault(
47             token,
48             victim,
49             accountant,
```

```

50         500,
51         "vUSDC",
52         "Victim Vault USDC",
53         SALT
54     );
55
56     // 4. The vault at the expected address is controlled by the attacker.
57     assertEq(deployedVault, expectedVault);
58     assertEq(HypoVault(payable(expectedVault)).manager(), attacker);
59 }
60 }

```

Recommended mitigation

Incorporate `msg.sender` into the salt before passing it to `cloneDeterministic`. This namespaces each caller's salt space so no two callers can ever collide and stop the annoying front running attempts:

```

1  function createVault(
2      address underlyingToken,
3      address manager,
4      IVaultAccountant accountant,
5      uint256 performanceFeeBps,
6      string memory symbol,
7      string memory name,
8      bytes32 salt
9  ) external returns (address payable vault) {
10     bytes32 effectiveSalt = keccak256(abi.encode(msg.sender, salt));
11     vault = payable(Clones.cloneDeterministic(hypoVaultReference, effectiveSalt));
12     // ...
13 }

```

[L-02] Performance Fee Bypassed by Cross-Address PnL Netting via Share Transfer

Description

`HypoVault` computes the performance fee independently per address at the moment of withdrawal fulfillment:

```

1  // HypoVault.sol L496-500
2  uint256 withdrawnBasis = (uint256(pendingWithdrawal.basis) *
3      _withdrawalEpochState.sharesFulfilled) / _withdrawalEpochState.sharesWithdrawn;
4  uint256 performanceFee = (uint256(
5      Math.max(0, int256(assetsToWithdraw) - int256(withdrawnBasis))
6  ) * performanceFeeBps) / 10_000;

```

The `Math.max(0, ...)` clamps negative PnL to zero - a losing withdrawal pays no fee, and that loss does not offset a profit recorded elsewhere. An entity that controls two addresses - one with unrealised profit and one with unrealised loss - can consolidate both positions into the losing address before withdrawing, netting the PnL inside a single `max()` call and eliminating the fee that would have been owed on the profit.

The mechanism is the ERC20 `transfer` / `transferFrom` override, which invokes `_transferBasis`. When the profitable address (lower basis per share) transfers its shares to the losing address (higher basis per share), `_transferBasis` adds the sender's actual basis to the recipient:

```

1  // HypoVault.sol L609-610
2  // reset basis to the lowest average between the sender and receiver to ensure performance
   fee is not deflated
3  userBasis[to] += Math.min(basisToTransfer, (userBasis[from] * amount) / toBalance);

```

The `Math.min` guard is intended to prevent basis inflation (a high-basis address inflating a low-basis recipient). It does not prevent the opposite: when the sender has a *lower* per-share basis than the recipient, `basisToTransfer`

$\min((\text{userBasis}[\text{to}] * \text{amount}) / \text{toBalance}, \text{Math.min}(\text{basisToTransfer} - \text{senderBasis}, \text{toBasis} - \text{senderBasis}))$, so `Math.min` resolves to `basisToTransfer` - the full sender basis is transferred, and the combined position's average basis rises to absorb the loss.

Concrete example (20% performance fee):

- Address A (profit): 100 shares, basis 1,000 USDC, receives 1,500 USDC, PnL +500 USDC - fee owed **100 USDC**
- Address B (loss): 100 shares, basis 2,000 USDC, receives 1,500 USDC, PnL -500 USDC - fee owed 0
- Separate exit total fee: **100 USDC**
- After A -> B transfer, total fee: **0**

After the transfer, Address B holds 200 shares, basis = 3,000 USDC, receives 3,000 USDC, and pays $\max(0, 3000 - 3000) * 20\% = 0$. The entire performance fee is eliminated.

Impact

Any user controlling two addresses - or cooperating with another user - can suppress performance fees proportionally to the size of offsetting positions. The fee model is therefore sybil-sensitive: a single economic actor split across N addresses pays, at best, 1/N of the fee they would owe if consolidated, and at worst zero if loss positions are available to absorb profits.

Recommended Mitigation

In `_transferBasis`, when the sender's per-share basis is lower than the recipient's, apply the fee on the profit delta at transfer time rather than deferring it to withdrawal. This crystallises earned fees when shares move between addresses.

[L-03] Permissionless executeWithdrawal can front-run user opt-out and force redeposit

Description `HypoVault` allows anyone to call `executeWithdrawal(user, epoch)` once an epoch is fulfilled, while `optOutOfRedeposit(epoch)` can only be called by the user. If a pending withdrawal has `shouldRedeposit = true`, this creates a race condition:

1. Withdrawal is queued with redeposit enabled.
2. Epoch is fulfilled.
3. User tries to opt out `optOutOfRedeposit`.
4. A third party front-runs with `executeWithdrawal(user, epoch)`.

At that point, execution uses the current stored flag and can route proceeds into redeposit before the user's opt-out is applied. This does not steal funds, but it removes user control over withdrawal destination for that epoch and can force an extra round-trip through deposit/withdraw flows.

Recommended mitigation Restrict `executeWithdrawal(user, epoch)` to the user (or approved operator), at least when `shouldRedeposit == true`. Alternatively, require explicit user authorization at execution time for redeposit-enabled withdrawals.

[I-01] Uninitialized feeWallet causes partial withdrawal DoS

Description

`HypoVault.initialize()` never assigns the `feeWallet` state variable, leaving it at its default value of `address(0)`:

```

1 // HypoVault.sol:226-245
2 function initialize(
3     address _underlyingToken,
4     address _manager,
5     IVaultAccountant _accountant,
6     uint256 _performanceFeeBps,
7     string memory _symbol,
8     string memory _name
9 ) external initializer {
10     __Ownable_init(_manager);
11     if (_performanceFeeBps > 10_000) revert PerformanceFeeTooHigh();
12     underlyingToken = _underlyingToken;
13     manager = _manager;
14     accountant = _accountant;
15     performanceFeeBps = _performanceFeeBps;
16     _internalSupply = 1_000_000;
17     symbol = _symbol;
18     name = _name;
19     // feeWallet is never set
20 }

```

`performanceFeeBps` can be set to any non-zero value at initialisation time. Whenever a user's withdrawal is profitable, `executeWithdrawal()` deducts a fee from the user's proceeds and transfers it to `feeWallet`:

```

1 // HypoVault.sol:498-525
2 uint256 performanceFee = (uint256(
3     Math.max(0, int256(assetsToWithdraw) - int256(withdrawnBasis))
4     ) * performanceFeeBps) / 10_000;
5
6 // ...
7
8 if (performanceFee > 0) {
9     assetsToWithdraw -= performanceFee;
10    // user loses the fee
11    SafeTransferLib.safeTransfer(underlyingToken, feeWallet, uint256(performanceFee));
12    // transfer reverts because feeWallet == 0
13 }

```

However, as in or case `feeWallet` is `address(0)` initially the transfer fails and the user can not `executeWithdrawal` timely.

Impact Two distinct failure modes arise depending on the underlying token:

- Reverts on `transfer(to = address(0))` - standard OpenZeppelin ERC-20. Every call to `executeWithdrawal` for a profitable position reverts. Withdrawal processing is **completely blocked** for all such users until the owner manually calls `setFeeWallet`.
- Allows `transfer(to = address(0))` - effectively burns the token. Performance fees are silently and permanently destroyed. The protocol collects zero revenue and the tokens are unrecoverable.

In both cases, since there is no enforced ordering between vault deployment and fee wallet configuration, and vaults are usable immediately after `createVault` returns, the vulnerable window begins at block 0 of the vault's life. Any vault deployed with `performanceFeeBps > 0` and used before `setFeeWallet` is called is affected.

Proof of Concept

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.28;
3
4 import "forge-std/Test.sol";
5 import {HypoVaultFactory} from "src/HypoVaultFactory.sol";
6 import {HypoVault} from "src/HypoVault.sol";

```

```

7  import {IVaultAccountant} from "src/interfaces/IVaultAccountant.sol";
8  import {ERC20Mock}          from "lib/openzeppelin-contracts-upgradeable/lib/openzeppelin-
    contracts/contracts/mocks/token/ERC20Mock.sol";
9
10 contract MockAccountant is IVaultAccountant {
11     uint256 public nav;
12     function setNAV(uint256 _nav) external { nav = _nav; }
13     function computeNAV(address, address, bytes memory) external view returns (uint256) {
14         return nav;
15     }
16 }
17
18 contract UninitializedFeeWalletTest is Test {
19     HypoVaultFactory factory;
20     HypoVault         vault;
21     ERC20Mock         token;
22     MockAccountant    accountant;
23
24     address manager = makeAddr("manager");
25     address user    = makeAddr("user");
26
27     function setUp() public {
28         token      = new ERC20Mock();
29         accountant = new MockAccountant();
30
31         address vaultImpl = address(new HypoVault());
32         factory = new HypoVaultFactory(vaultImpl);
33
34         // Deploy vault with 10% performance fee -- feeWallet is NOT set.
35         address payable vaultAddr = factory.createVault(
36             address(token),
37             manager,
38             IVaultAccountant(address(accountant)),
39             1_000, // 10% performance fee
40             "vTKN",
41             "Vault Token",
42             bytes32(uint256(1))
43         );
44         vault = HypoVault(vaultAddr);
45
46         // Confirm feeWallet is address(0).
47         assertEq(vault.feeWallet(), address(0));
48     }
49
50     function test_withdrawalDosWhenFeeWalletUnset() public {
51         // 1. User deposits 1000 tokens.
52         token.mint(user, 1_000e18);
53         vm.startPrank(user);
54         token.approve(address(vault), type(uint256).max);
55         vault.requestDeposit(1_000e18);
56         vm.stopPrank();
57
58         // 2. Manager fulfills the deposit at a 1:1 NAV.
59         accountant.setNAV(1_000e18);
60         vm.prank(manager);
61         vault.fulfillDeposits(1_000e18, "");
62
63         // 3. User claims shares.
64         vault.executeDeposit(user, 0);
65
66         // 4. User requests a withdrawal.
67         uint128 shares = uint128(vault.balanceOf(user));
68         vm.prank(user);
69         vault.requestWithdrawal(shares);

```

```

70
71     // 5. NAV grows -- position is now profitable; performance fee will be > 0.
72     accountant.setNAV(2_000e18);
73
74     // 6. Manager fulfills the withdrawal.
75     token.mint(address(vault), 2_000e18);
76     vm.prank(manager);
77     vault.fulfillWithdrawals(shares, type(uint256).max, "");
78
79     // 7. executeWithdrawal reverts because SafeTransferLib sends fee to address(0)
80     //     and the underlying ERC-20 reverts on transfer to the zero address.
81     vm.expectRevert();
82     vault.executeWithdrawal(user, 0);
83
84     // User's profitable withdrawal is permanently stuck.
85 }
86 }

```

Recommended mitigation

Add a `_feeWallet` parameter to `initialize()` and assign it immediately:

In HypoVault:

```

1  function initialize(
2      address _underlyingToken,
3      address _manager,
4      IVaultAccountant _accountant,
5      uint256 _performanceFeeBps,
6      address _feeWallet,          // added
7      string memory _symbol,
8      string memory _name
9  ) external initializer {
10     __Ownable_init(_manager);
11
12     if (_performanceFeeBps > 10_000) revert PerformanceFeeTooHigh();
13     if (_performanceFeeBps > 0 && _feeWallet == address(0)) revert InvalidFeeWallet(); //
        added
14
15     underlyingToken = _underlyingToken;
16     manager         = _manager;
17     accountant      = _accountant;
18     performanceFeeBps = _performanceFeeBps;
19     feeWallet        = _feeWallet;          // added
20     _internalSupply  = 1_000_000;
21     symbol           = _symbol;
22     name             = _name;
23 }

```

In HypoVaultFactory:

```

1  /// @notice Creates a new HypoVault instance.
2  /// @param underlyingToken The token used to denominate deposits and withdrawals
3  /// @param manager The account authorized to execute deposits, withdrawals, and make
4  /// @param accountant The contract that reports the net asset value of the vault
5  /// @param performanceFeeBps The performance fee, in basis points, taken on each
6  /// @param symbol The symbol of the share token
7  /// @param name The name of the share token
8  /// @return vault The address of the newly created vault
9  function createVault(
10     address underlyingToken,
11     address manager,

```

```
12     IVaultAccountant accountant,  
13     uint256 performanceFeeBps,  
14     address feeWallet,  
15     string memory symbol,  
16     string memory name,  
17     bytes32 salt  
18 ) external returns (address payable vault) {  
19     // NOTE shouldn't msg.sender be added to the salt.  
20     vault = payable(Clones.cloneDeterministic(hypoVaultReference, salt));  
21  
22     // NOTE feeWallet is not set in the vault, should it be?  
23     HypoVault(vault).initialize(  
24         underlyingToken,  
25         manager,  
26         accountant,  
27         performanceFeeBps,  
28         feeWallet,  
29         symbol,  
30         name  
31     );
```

Propagate `_feeWallet` through `HypoVaultFactory.createVault()` accordingly and add the same non-zero guard to `setFeeWallet()` to prevent the zero-address state from being re-introduced after deployment.